

# OOP reflection report

Group 3: B123040032 李承諺 B123040039 陳彥維 B123040036 徐子皓

---

## Part 2: Frozen Lake

The goal requires us to achieve a relatively high success rate with the limit of a maximum of 100 steps per episode. We consider this an impossible task because in this environment with high randomness, it is very difficult to reach the end without falling into holes. To reach the end fast, the risk of falling into holes must be taken. However, if one moves safely, the step limit of 100 steps will be exhausted very quickly.

We tried the python package “Optuna” to help us optimize the parameters of the Q-Learning Agent. It works, but not much. We tried adding custom reward systems, such as a step penalty, an attraction to the end goal, and hole danger assessment. None of them significantly boosted the success rate. We then tried to extract the environment model and use dynamic programming to find the optimal path. The success rate became more stable, but it still did not improve.

At this point, we thought we had tried everything we could, so we started to question if the requirements were unreasonable. Therefore, we wrote another program so we could play the game ourselves and observe the environment. It turned out that even under our control, the slippery tiles drove everyone out with their randomness. We then defined a fixed mapping table for the agent to let it follow the path we defined, but the best success rate was stuck at around 62%.

In summary, our conclusion is that 62% is the highest success rate achievable under the limit of 100 steps. To consistently reach a 70% success rate, the limitation must be relaxed to 115 steps.

## Part 3: Choose Your Own Adventure

We want to create a game in which users can control a robot, overcome obstacles, and ultimately win the game. We therefore referenced the "Dungeon Crawler" environment and added moving enemies, traps, and a door requiring a key that the player must pick up.

We started with Q-Learning and the initial observation space to train the agent. In short, the result was a disaster. The agent did not know how to dodge enemies, and frequently wandered around and lost health. We then optimized the observation space so the agent could see the entire map, including the locations of enemies, traps, the door, and the key. Turns out, the performance still disappointed us; the agent tried to dodge, but still couldn't pathfind to the end consistently.

This is where things changed: we then used DQN as our agent, incorporating a CNN as the vision integration network, and adjusted the reward function logics. This resulted in an outstanding performance. The agent began to play chasing and running with the enemies, even predicting the actions of multiple monsters to avoid being cornered. Observing the agent's gameplay from a God's-eye view was quite fascinating, as it moved like an experienced player controlling the robot. After that, we were determined to optimize the DQN agent, so we implemented the Double DQN agent, but the performance did not outperform DQN.

In the end, we tried PPO. However, it performed worse than the two agents we just mentioned. After trials and errors, we found that PPO updates slower than DQN does, and increasing the epoch count is also not a practical solution since it might lead to overfitting. At this point, we decided to abandon the agent since the performance of the agent is not the evaluation criterion.

# Overall Reflection

This was the first time the three of us engaged in a collaborative project using a Git repository. Honestly, we had no idea how to efficiently work on multiple branches with similar or completely different tasks. Merging the works was the most annoying problem we encountered; we tried merging the code manually, asking generative AI, and simply letting AI take over the entire merging task. We did learn something, but we are still figuring out a better solution. Besides that, in a project of this scale, keeping up with other collaborators' tasks was perplexing, and AI tools helped a lot with that, as well as with debugging and generating ideas, among other things.